

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

PROJECT REPORT

ON

ADVANCED DATABASE MANAGEMENT SYSTEM

PROJECT TOPIC

DISTRIBUTED STUDENT DATABASE MANAGEMENT SYSTEM

Submitted By:

Name: Anshu

UID: 26MCD10027

Section: 26MCD-1

Group:A

SubmittedTo:

Ms. Winky Bhatia

(E18463)

University Institute of Computing

ChandigarhUniversity,Gharuan,Mohali

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"Distributed Student Database Management System"** submitted by **Anshu (UID: 26MCD10027)**, a student of MCA Data Science at the University Institute of Computing, is a bonafide record of the project work carried out under the guidance and supervision of **Ms. Winky Bhatia**.

The project has been developed as part of the academic requirements for the study of Advanced Database Management Systems (ADBMS). It demonstrates the basic concepts of distributed database organization, data fragmentation, data allocation, and database operations through a student database management application.

Project Supervisor:	**Head of Department:**
Ms. Winky Bhatia	Dr. Amanpreet Kaur Sandhu,
(E18463)	HOD – MCA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Dr. Amanpreet Kaur Sandhu, Head of the Department (MCA)**, University Institute of Computing, for providing me with the opportunity and academic support to undertake this project entitled **"Distributed Student Database Management System."**

I am especially thankful to my project supervisor, **Ms. Winky Bhatia**, for her valuable guidance, continuous encouragement, and constructive suggestions throughout the development of this project. Her support and guidance helped me understand the fundamental concepts of Advanced Database Management Systems (ADBMS) and their practical application in database management.

I would also like to extend my sincere thanks to all the faculty members of the department for their cooperation and encouragement. Finally, I am grateful to my family and friends for their constant motivation and support, which helped me complete this project successfully.

This project has been a valuable learning experience that has enhanced my understanding of database management, data organization, and the basic principles of distributed database systems.

Anshu
26MCD10027
MCA Data Science

INDEX

S. No.	Contents	Page No.
1.	Bonafide Certificate	2
2.	Acknowledgement	3
3.	Index	4
4.	Introduction	5
5.	Objectives and Problem Statement	6
6.	Tools and Technologies Used	7-8
7.	System Design and Workflow	8-9
8.	Database Design and SQL Operations	10-16
9.	Implementation and Modules	16-18
10.	Testing and Results	18-19
11.	Advantages and Limitations	19
12.	Future Scope and Conclusion	20
13.	References	20

1. INTRODUCTION

1.1 Project Overview

The Distributed Student Database Management System is a database project designed to store and manage student information associated with different campuses.

Educational institutions need to maintain student details such as roll number, name, course, campus, and marks. Managing these records manually can be time-consuming and may lead to errors.

This project uses Oracle Free SQL to execute SQL and PL/SQL programs for student record management. The student information is stored in a table named CAMPUS_STUDENT.

The system supports basic database operations, including inserting new records, displaying student details, searching for a student, updating marks, deleting records, and retrieving campus-wise information.

SQL is used to create tables and perform database operations. PL/SQL is used to demonstrate procedural programming, including variables, cursors, and exception handling.

The project provides practical experience in database management and demonstrates how student records can be organized and processed efficiently.

Scope of the current implementation: The project stores student records in one database table and identifies campuses through a CAMPUS column. It demonstrates campus-wise record management but does not yet implement separate physical databases at different campuses.

1.2 Need for the Project

Maintaining student records manually can create several problems.

1. Searching for a particular student's information may take time.
2. Duplicate student records can create confusion.
3. Updating marks and other information manually can lead to mistakes.
4. Preparing campus-wise student lists requires additional effort.
5. Calculating average marks and student counts manually is time-consuming.
6. Managing a growing number of student records becomes difficult.

The proposed system provides a structured way to store and manage student information.

It uses SQL queries to retrieve and modify data and PL/SQL programs to perform procedural operations and handle errors.

The system helps reduce manual work and improves the organization of student records.

2. OBJECTIVES AND PROBLEM STATEMENT

2.1 Problem Definition

Educational institutions maintain information about students enrolled in different courses and associated with different campuses.

When student records are maintained manually, searching, updating, and organizing information can become difficult. There is also a possibility of incorrect entries, duplicate roll numbers, and errors during calculations.

The problem addressed by this project is to create a database-based system that stores student details in a structured format and supports common record management operations.

The project uses Oracle Free SQL, SQL, and PL/SQL to manage student information and generate useful results.

2.2 Project Objectives

The main objectives of the project are:

1. To create a database table for storing student information.
2. To insert student records using SQL.
3. To retrieve and display all student records.
4. To search for students using their roll numbers.
5. To update student marks.
6. To delete selected student records.
7. To display student records according to campus.
8. To calculate the number of students at each campus.
9. To calculate average, maximum, and minimum marks.
10. To implement PL/SQL programs for student record management.
11. To demonstrate cursor operations.
12. To implement exception handling for database errors.
13. To validate student marks using database constraints.
14. To develop practical knowledge of SQL and PL/SQL.
15. To provide a foundation for a future distributed student database system.

3. TOOLS AND TECHNOLOGIES USED

3.1 Oracle Free SQL

Oracle Free SQL is the primary environment used for executing SQL statements and PL/SQL programs in this project.

It is used to create the student table, insert records, retrieve data, update marks, delete records, and execute procedural database programs.

3.2 SQL (Structured Query Language)

SQL is a language used to communicate with a relational database.

It allows users to create tables and perform operations on the stored information.

The project uses the following SQL commands:

- CREATE TABLE to create the student table.
- INSERT to add student records.
- SELECT to retrieve records.
- UPDATE to modify student information.
- DELETE to remove selected records.
- GROUP BY to generate campus-wise summaries.
- Aggregate functions to calculate student statistics.

3.3 PL/SQL

PL/SQL is Oracle's procedural extension of SQL.

It allows SQL statements to be combined with variables, conditional statements, loops, cursors, and exception handling.

In this project, PL/SQL is used to insert student records through a program, display records using a cursor, validate marks, and handle database errors.

3.4 Software Requirements

S. No.	Software/Technology	Purpose
1	Oracle Free SQL	Executing SQL and PL/SQL
2	SQL	Creating and managing student records
3	PL/SQL	Implementing procedural database operations
4	Web Browser	Accessing the Oracle Free SQL environment

3.5 Hardware Requirements

S. No.	Hardware	Requirement
1	Computer or Laptop	Required for project work
2	Processor	Capable of running a web browser
3	RAM	4 GB or more recommended
4	Internet Connection	Required if using an online Oracle SQL environment

4. SYSTEM DESIGN AND WORKFLOW

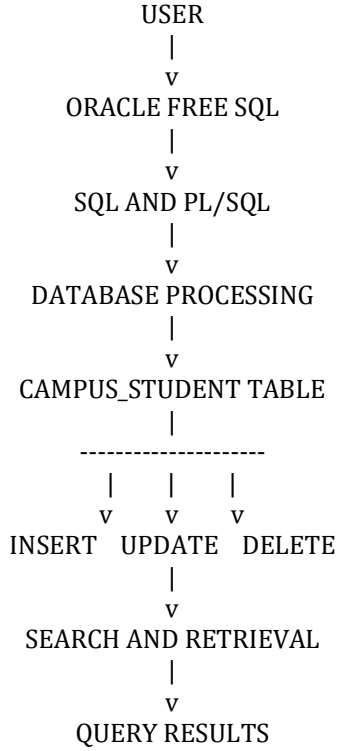
4.1 System Architecture

The system follows a basic database application architecture.

The user writes and executes SQL or PL/SQL statements through Oracle Free SQL. The database processes the statements and stores or retrieves student information.

The results are displayed in the SQL output area.

System Architecture Diagram

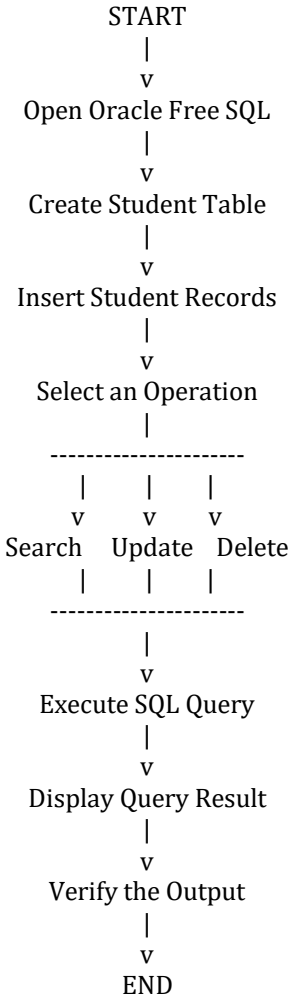


4.2 Application Workflow

The application follows these steps:

1. The user opens Oracle Free SQL.
2. The user writes and executes the table creation statement.
3. Student records are inserted into the table.
4. The user executes SQL queries to retrieve student information.
5. The user searches for records using roll numbers or campus names.
6. Update statements modify student information when required.
7. Delete statements remove selected records.
8. Aggregate queries calculate statistics.
9. PL/SQL programs perform procedural operations.
10. The output is checked to verify that the operations have worked correctly.

4.3 Flowchart



5. DATABASE DESIGN AND SQL OPERATIONS

Table Structure

The project uses a table named CAMPUS_STUDENT to store student details.

The table contains five columns.

Column Name	Data Type	Description
ROLL_NO	NUMBER	Unique roll number of the student
NAME	VARCHAR2(50)	Name of the student
COURSE	VARCHAR2(50)	Course name
CAMPUS	VARCHAR2(30)	Campus associated with the student
MARKS	NUMBER	Student marks between 0 and 100

Database Constraints

The table uses the following constraints:

Primary Key: The ROLL_NO column uniquely identifies each student.

NOT NULL: The NAME column cannot contain a null value.

CHECK Constraint: The MARKS column accepts values between 0 and 100.

These constraints help maintain the validity and consistency of the stored records.

5.1 Create Student Table

Program:

```
CREATE TABLE CAMPUS_STUDENT (
  ROLL_NO NUMBER PRIMARY KEY,
  NAME VARCHAR2(50) NOT NULL,
  COURSE VARCHAR2(50),
  CAMPUS VARCHAR2(30),
  MARKS NUMBER CHECK (MARKS BETWEEN 0 AND 100)
);
```

Output:

Table created successfully.

5.2 Insert Student Records

Program:

```
INSERT INTO CAMPUS_STUDENT
VALUES (101, 'Anshu', 'MCA Data Science', 'Ludhiana', 85);

INSERT INTO CAMPUS_STUDENT
VALUES (102, 'Aman', 'MCA Data Science', 'Chandigarh', 88);

INSERT INTO CAMPUS_STUDENT
VALUES (103, 'Simran', 'MCA', 'Mohali', 91);

INSERT INTO CAMPUS_STUDENT
VALUES (104, 'Rahul', 'MCA Data Science', 'Ludhiana', 82);

INSERT INTO CAMPUS_STUDENT
VALUES (105, 'Neha', 'MCA', 'Chandigarh', 79);

COMMIT;
```

Output:

```
5 rows inserted.
Commit complete.
```

5.3 Insert Student Using PL/SQL Exception Handling

Program:

```
DECLARE
  V_ROLL_NO NUMBER := 106;
  V_NAME VARCHAR2(50) := 'Karan';
  V_COURSE VARCHAR2(50) := 'MCA Data Science';
  V_CAMPUS VARCHAR2(30) := 'Mohali';
  V_MARKS NUMBER := 87;
BEGIN
  INSERT INTO CAMPUS_STUDENT
    (ROLL_NO, NAME, COURSE, CAMPUS, MARKS)
  VALUES
    (V_ROLL_NO, V_NAME, V_COURSE, V_CAMPUS, V_MARKS);

  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Student inserted successfully.');

EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
```

```
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Roll number already exists.');

   WHEN OTHERS THEN
      ROLLBACK;
      DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Output:

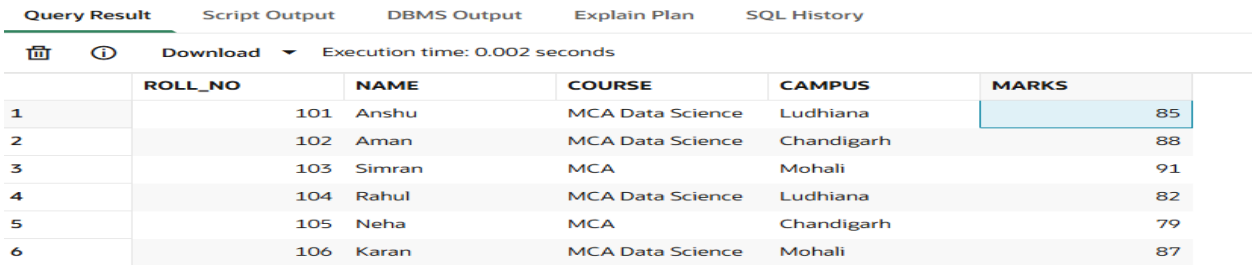
Student inserted successfully.

5.4 Display All Student Records

Program:

```
SELECT *
FROM CAMPUS_STUDENT
ORDER BY ROLL_NO;
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.002 seconds

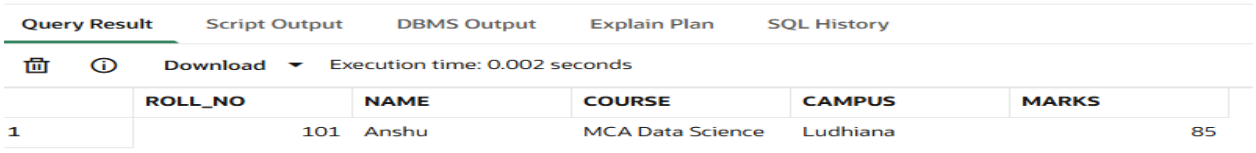
	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	101	Anshu	MCA Data Science	Ludhiana	85
2	102	Aman	MCA Data Science	Chandigarh	88
3	103	Simran	MCA	Mohali	91
4	104	Rahul	MCA Data Science	Ludhiana	82
5	105	Neha	MCA	Chandigarh	79
6	106	Karan	MCA Data Science	Mohali	87

5.5 Search Student by Roll Number

Program:

```
SELECT *
FROM CAMPUS_STUDENT
WHERE ROLL_NO = 101;
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.002 seconds

	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	101	Anshu	MCA Data Science	Ludhiana	85

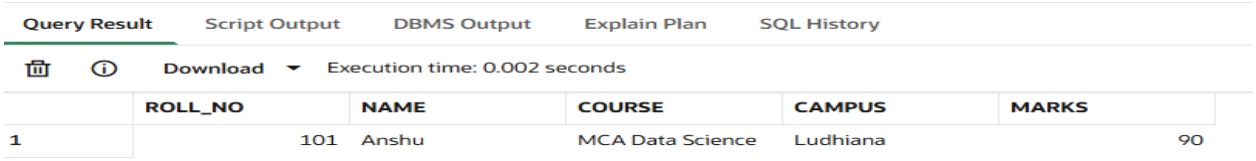
5.6 Update Student Marks

Program:

```
UPDATE CAMPUS_STUDENT
SET MARKS = 90
WHERE ROLL_NO = 101;

COMMIT;
SELECT *
FROM CAMPUS_STUDENT
WHERE ROLL_NO = 101;
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.002 seconds

	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	101	Anshu	MCA Data Science	Ludhiana	90

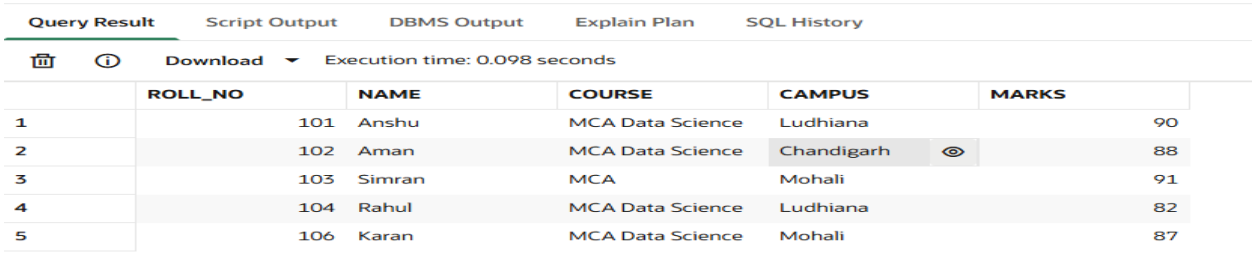
5.7 Delete Student Record

Program:

```
DELETE FROM CAMPUS_STUDENT
WHERE ROLL_NO = 105;

COMMIT;
SELECT *
FROM CAMPUS_STUDENT;
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.098 seconds

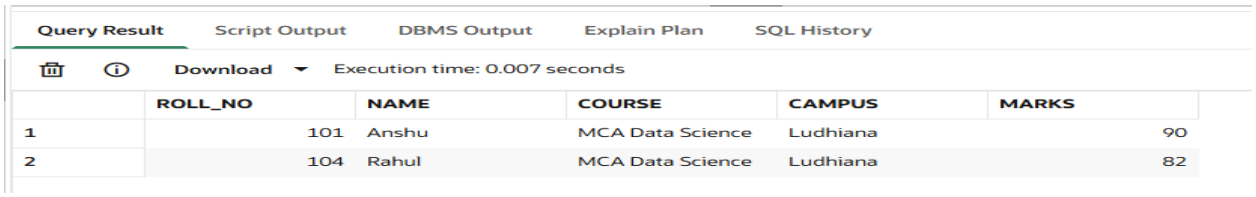
	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	101	Anshu	MCA Data Science	Ludhiana	90
2	102	Aman	MCA Data Science	Chandigarh	88
3	103	Simran	MCA	Mohali	91
4	104	Rahul	MCA Data Science	Ludhiana	82
5	106	Karan	MCA Data Science	Mohali	87

5.8 Display Students Campus-Wise

Program 1:

```
SELECT *
FROM CAMPUS_STUDENT
WHERE CAMPUS = 'Ludhiana';
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

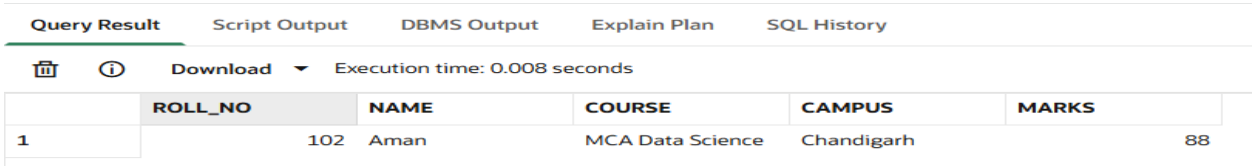
Download | Execution time: 0.007 seconds

	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	101	Anshu	MCA Data Science	Ludhiana	90
2	104	Rahul	MCA Data Science	Ludhiana	82

Program 2:

```
SELECT *
FROM CAMPUS_STUDENT
WHERE CAMPUS = 'Chandigarh';
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

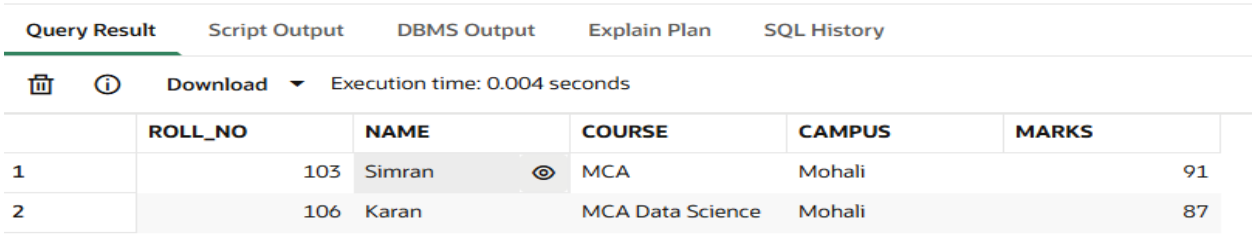
Download | Execution time: 0.008 seconds

	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	102	Aman	MCA Data Science	Chandigarh	88

Program 3:

```
SELECT *
FROM CAMPUS_STUDENT
WHERE CAMPUS = 'Mohali';
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.004 seconds

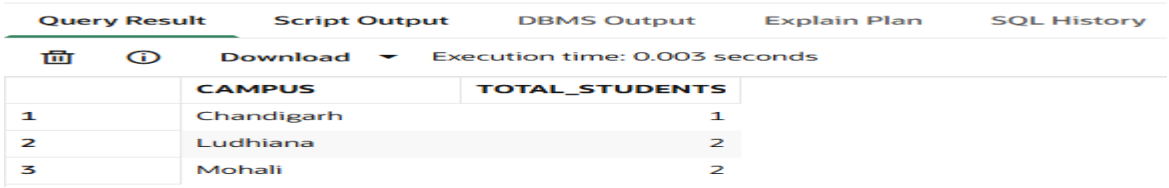
	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	103	Simran	MCA	Mohali	91
2	106	Karan	MCA Data Science	Mohali	87

5.9 Count Students at Each Campus

Program:

```
SELECT
   CAMPUS,
   COUNT(*) AS TOTAL_STUDENTS
FROM CAMPUS_STUDENT
GROUP BY CAMPUS
ORDER BY CAMPUS;
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.003 seconds

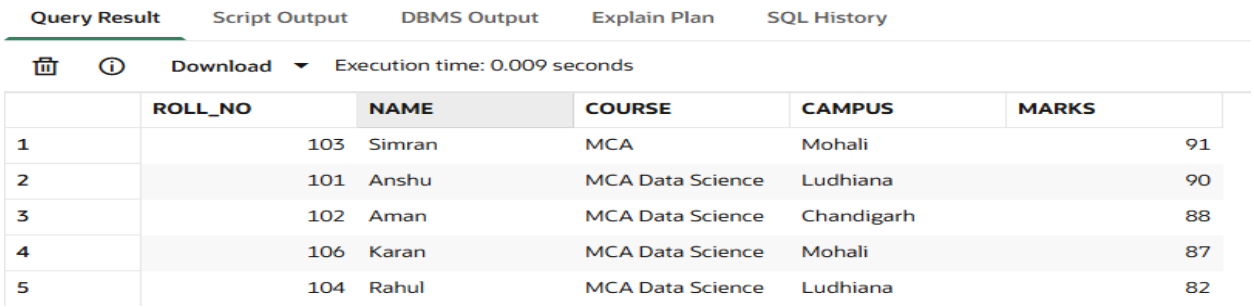
	CAMPUS	TOTAL_STUDENTS
1	Chandigarh	1
2	Ludhiana	2
3	Mohali	2

5.10 Display Students Scoring More Than 80 Marks

Program:

```
SELECT *
FROM CAMPUS_STUDENT
WHERE MARKS > 80
ORDER BY MARKS DESC;
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan | SQL History

Download | Execution time: 0.009 seconds

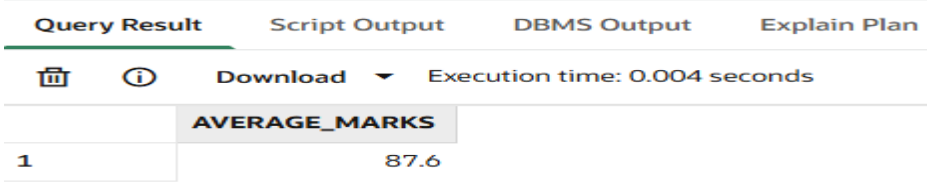
	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	103	Simran	MCA	Mohali	91
2	101	Anshu	MCA Data Science	Ludhiana	90
3	102	Aman	MCA Data Science	Chandigarh	88
4	106	Karan	MCA Data Science	Mohali	87
5	104	Rahul	MCA Data Science	Ludhiana	82

5.11 Calculate Average Marks

Program:

```
SELECT
   AVG(MARKS) AS AVERAGE_MARKS
FROM CAMPUS_STUDENT;
```

Output:



Query Result | Script Output | DBMS Output | Explain Plan

Download | Execution time: 0.004 seconds

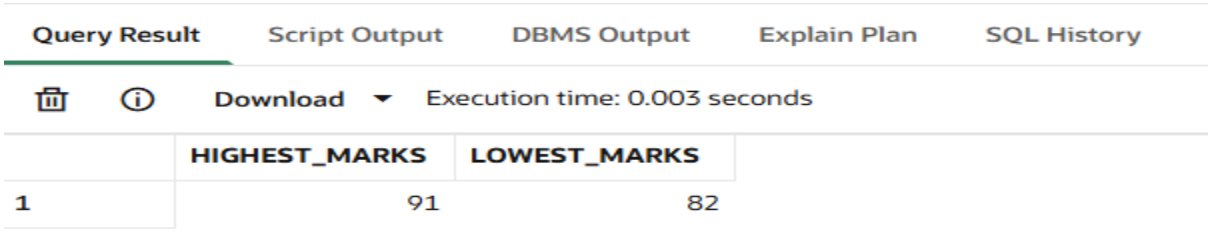
	AVERAGE_MARKS
1	87.6

5.12 Find Highest and Lowest Marks

Program:

```
SELECT
   MAX(MARKS) AS HIGHEST_MARKS,
   MIN(MARKS) AS LOWEST_MARKS
FROM CAMPUS_STUDENT;
```

Output:



	HIGHEST_MARKS	LOWEST_MARKS
1	91	82

6. IMPLEMENTATION AND MODULES

6.1 Display Student Records Using a Cursor

Program:

```
DECLARE
   CURSOR STUDENT_CURSOR IS
      SELECT ROLL_NO, NAME, MARKS
      FROM CAMPUS_STUDENT
      ORDER BY ROLL_NO;

   V_ROLL_NO CAMPUS_STUDENT.ROLL_NO%TYPE;
   V_NAME CAMPUS_STUDENT.NAME%TYPE;
   V_MARKS CAMPUS_STUDENT.MARKS%TYPE;

BEGIN
   OPEN STUDENT_CURSOR;

   LOOP
      FETCH STUDENT_CURSOR
      INTO V_ROLL_NO, V_NAME, V_MARKS;

      EXIT WHEN STUDENT_CURSOR%NOTFOUND;

      DBMS_OUTPUT.PUT_LINE(
         'Roll No: ' || V_ROLL_NO ||
         ', Name: ' || V_NAME ||
         ', Marks: ' || V_MARKS
      );
   END LOOP;
```

```
  CLOSE STUDENT_CURSOR;

EXCEPTION
  WHEN OTHERS THEN
    IF STUDENT_CURSOR%ISOPEN THEN
      CLOSE STUDENT_CURSOR;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Output:

```
Roll No: 101, Name: Anshu, Marks: 90
Roll No: 102, Name: Aman, Marks: 88
Roll No: 103, Name: Simran, Marks: 91
Roll No: 104, Name: Rahul, Marks: 82
Roll No: 106, Name: Karan, Marks: 87
```

6.2 Insert Student with Marks Validation

Program:

```
DECLARE
  V_ROLL_NO NUMBER := 107;
  V_NAME VARCHAR2(50) := 'Priya';
  V_COURSE VARCHAR2(50) := 'MCA Data Science';
  V_CAMPUS VARCHAR2(30) := 'Ludhiana';
  V_MARKS NUMBER := 95;

BEGIN
  IF V_MARKS < 0 OR V_MARKS > 100 THEN
    RAISE_APPLICATION_ERROR(
      -20001,
      'Marks must be between 0 and 100.'
    );
  END IF;

  INSERT INTO CAMPUS_STUDENT
    (ROLL_NO, NAME, COURSE, CAMPUS, MARKS)
  VALUES
    (V_ROLL_NO, V_NAME, V_COURSE, V_CAMPUS, V_MARKS);

  COMMIT;

  DBMS_OUTPUT.PUT_LINE('Student added successfully.');

EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Roll number already exists.');

  WHEN OTHERS THEN
```

```
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
```

Output:

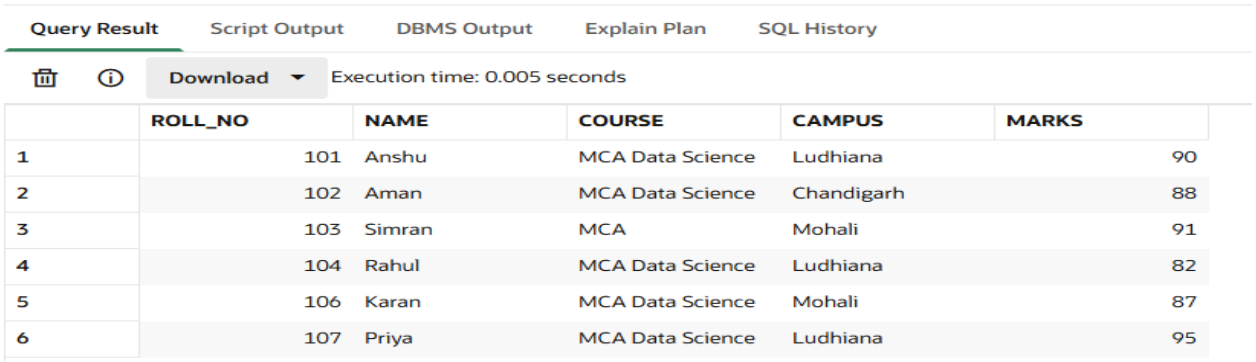
Student added successfully.

6.3 Display Final Student Records

Program:

```
SELECT
  ROLL_NO,
  NAME,
  COURSE,
  CAMPUS,
  MARKS
FROM CAMPUS_STUDENT
ORDER BY ROLL_NO;
```

Output:



	ROLL_NO	NAME	COURSE	CAMPUS	MARKS
1	101	Anshu	MCA Data Science	Ludhiana	90
2	102	Aman	MCA Data Science	Chandigarh	88
3	103	Simran	MCA	Mohali	91
4	104	Rahul	MCA Data Science	Ludhiana	82
5	106	Karan	MCA Data Science	Mohali	87
6	107	Priya	MCA Data Science	Ludhiana	95

7. TESTING AND RESULTS

7.1 Testing

Testing was performed to check whether the Distributed Student Database Management System works correctly. Different SQL queries and PL/SQL programs were used to test the system.

The following operations were tested:

1. **Table Creation:** The student table was created successfully.
2. **Data Insertion:** Student records were inserted into the table.
3. **Data Retrieval:** Student details were displayed using the SELECT statement.
4. **Student Search:** Student records were searched using roll numbers.

5. **Data Update:** Student marks were updated successfully.
6. **Data Deletion:** A student record was deleted successfully.
7. **Campus-Wise Search:** Student records were displayed according to their campus.
8. **Aggregate Functions:** Average, highest, and lowest marks were calculated.
9. **Cursor Operation:** Student records were retrieved using a PL/SQL cursor.
10. **Exception Handling:** PL/SQL exception handling was used to handle errors during insertion.

7.2 Results

The project successfully demonstrated student record management using SQL and PL/SQL in Oracle Free SQL.

The system performed insertion, searching, updating, deletion, campus-wise filtering, and marks calculation. It also demonstrated cursor processing and exception handling.

The project helped in understanding database operations and the management of student information through SQL queries.

8. ADVANTAGES AND LIMITATIONS

8.1 Advantages

1. **Easy Data Management:** Student information can be stored and managed easily.
2. **Fast Searching:** Student records can be searched using roll numbers.
3. **Easy Updating:** Student marks and other details can be updated.
4. **Campus-Wise Records:** Students can be filtered according to their campus.
5. **Data Validation:** Constraints help prevent invalid marks from being entered.
6. **Error Handling:** PL/SQL exception handling helps manage errors.
7. **Reduced Manual Work:** SQL queries make student record management easier.
8. **Data Analysis:** Aggregate functions help calculate average, highest, and lowest marks.

8.2 Limitations

1. The current implementation uses a single database table.
2. Separate databases at different campuses have not been connected.
3. The project does not include a graphical user interface.
4. User login and role-based access control have not been implemented.
5. The system is a basic academic implementation and may require additional features for large institutions.

9. FUTURE SCOPE AND CONCLUSION

9.1 Future Scope

The project can be improved in the following ways:

1. **Distributed Database Connectivity:** Separate databases can be created for different campuses and connected using distributed database technologies.
2. **User Interface:** A web-based application can be developed to make the system easier to use.
3. **Login System:** Authentication can be added for administrators, teachers, and students.
4. **Additional Student Details:** Email, phone number, attendance, and other academic information can be included.
5. **Report Generation:** The system can generate student performance reports.
6. **Security:** User permissions and additional security measures can be implemented.
7. **Backup and Recovery:** Backup and recovery facilities can be added to protect student information.

9.2 Conclusion

The Distributed Student Database Management System project demonstrates how student records can be managed using SQL and PL/SQL in Oracle Free SQL.

The project includes table creation, insertion, searching, updating, deletion, campus-wise filtering, aggregate functions, cursor operations, and exception handling.

It provides practical knowledge of database design and database operations. The project also helps in understanding how student information can be organized and retrieved efficiently.

Although the current implementation uses a single database table, it provides a foundation for developing a complete distributed database system in the future.

10. REFERENCES

1. Oracle Documentation, SQL Language Reference.
2. Oracle Documentation, PL/SQL Language Reference.
3. Oracle Free SQL, used for writing and executing SQL and PL/SQL statements.
4. Database Management System course notes and study materials.
5. SQL and PL/SQL practical exercises completed during the project.